



Transilvania
University
of Brasov



Transilvania
University
of Brasov

FACULTY OF ELECTRICAL ENGINEERING
AND COMPUTER SCIENCE

Department of Electronics and Computers
Study programme: Applied Electronics

VERIFICATION PROJECT

Color Sensor

Table of Contents

1	Digital design of the circuit	3
1.1	Block diagram	3
1.2	Description of the general DUT functionality	3
1.3	DUT interfaces	4
1.4	List of DUT functionalities	4
1.5	Schematic diagram	7
1.6	Waveforms	7
1.7	Register table	8
2	Verification of the circuit digital design	12
2.1	Verification environment architecture	12
2.1.1	Driver for the APB interface	13
2.1.2	Driver for the I2C interface	14
2.1.3	Scoreboard / reference model	15
2.1.4	Monitor for the APB interface	15
2.1.5	Monitor for the I2C interface	16
2.1.6	Transaction for the APB interface	17
2.1.7	Transaction for the I2C interface	17
2.1.8	Agent for the APB interface	17
2.1.9	Agent for the I2C interface	18
2.1.10	Coverage collector for the I2C interface	18
2.1.11	Coverage collector for the scoreboard / reference model	18
2.2	List of elements to be verified	19
2.3	Functional coverage elements	20
2.4	Verification scenarios and tests	21
2.5	Results obtained	21

1 DIGITAL DESIGN OF THE CIRCUIT

Block diagram

Description of the general DUT functionality

DUT interfaces

List of DUT functionalities

Schematic diagram

Waveforms

Register table

1.1 BLOCK DIAGRAM

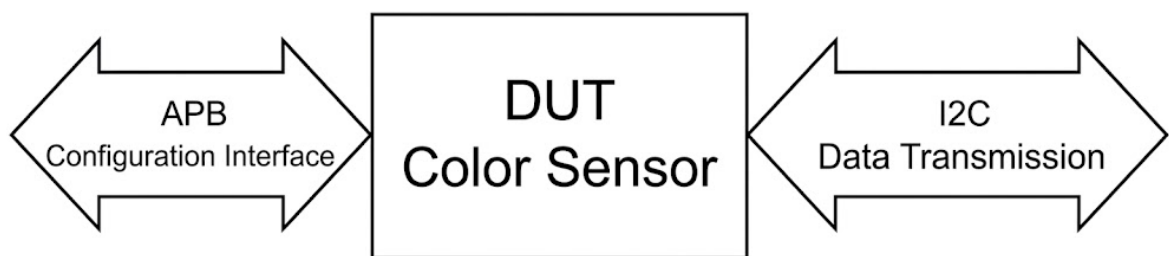


Figure 1. The interfaces of a DUT acting as a color sensor

1.2 DESCRIPTION OF THE GENERAL DUT FUNCTIONALITY

- Generates 16-bit color values (Red, Green, Blue, Clear, Infrared).
- Through the APB protocol, the sensor parameters can be configured, and the raw values can also be read through registers specific to each color channel.
- It has a status register that indicates communication errors on the I2C interface.
- The sensor has only a single operating mode: continuous.
- It allows data transmission on the I2C interface in Little Endian or Big Endian.
- The I2C clock frequency (SCL) can be configured.

1.3 DUT INTERFACES

APB Interface			
Signal name	Number of bits	Direction	Description
paddr	5	Input	APB addr bus
psel	1	Input	select
penable	1	Input	enable
pwrite	1	Input	APB write access - HIGH / read - LOW
pwrdata	32	Input	APB write data
prdata	32	Output	Read data
pready	1	Output	ack
pslverr	1	Output	slave error (HIGH - error, LOW - success)

Table 1. The signals of the APB interface

Table 2. The signals of the I2C interface

I2C Interface			
Signal name	Number of bits	Direction	Description
SCL	1	Output	Serial clock
SDA	1	I/O	Serial data

1.4 LIST OF DUT FUNCTIONALITIES

1. The module simulates a color sensor that generates 16-bit values for the following channels:
 - a. Red
 - b. Green
 - c. Blue

- d. Clear
 - e. Infrared
- 2. The raw values of each channel are stored in dedicated registers and are accessible through the APB interface.
- 3. The module allows configuring the sensor parameters through the APB interface:
 - a. Selection of the data transmission order: Little Endian or Big Endian;
 - b. The I2C address that the sensor must access in order to transmit the data;
 - c. The I2C clock frequency (SCL):
 - i. 00 -> 100 kHz (Standard mode); in simulation it is 10 MHz
 - ii. 01 -> 400 kHz (Fast mode); in simulation it is 41.66 MHz
 - iii. 10 -> 1 MHz (Fast mode plus); in simulation it is 125 MHz
 - iv. 11 -> 3.4 MHz (High-speed mode); in simulation it is 250 MHz
- 4. The LFSR polynomials for each color channel:
 - a. Clear channel -> $x^{14} + x^5 + x^3 + x + 1$
 - b. Red channel -> $x^{11} + x^9 + 1$
 - c. Green channel -> $x^{13} + x^4 + x^3 + x + 1$
 - d. Blue channel -> $x^{12} + x^6 + x^4 + x + 1$
 - e. Infrared channel -> $x^8 + x^6 + x^5 + x^4 + 1$
- 5. The module includes a status register that:
 - a. Indicates communication errors over the I2C interface (Detected error: NACK);
 - b. Provides information about the sensor state (sensor active/inactive, busy/free);
- 6. The sensor supports only a single operating mode, continuous. Whenever the color values are updated, they will be transmitted on the I2C interface.
- 7. Data can be transmitted on the I2C interface in the format:
 - a. Little Endian: the least significant byte is transmitted first;
 - b. Big Endian: the most significant byte is transmitted first;
- 8. The endianness mode is configured through the configuration register.
- 9. Through an APB command, the sensor can be turned on/off (default is off), and the color channels can be individually disabled. When a channel is off, only values of 1 will be transmitted on the I2C interface for that channel (the communication lines are held at VDD / pull-up).

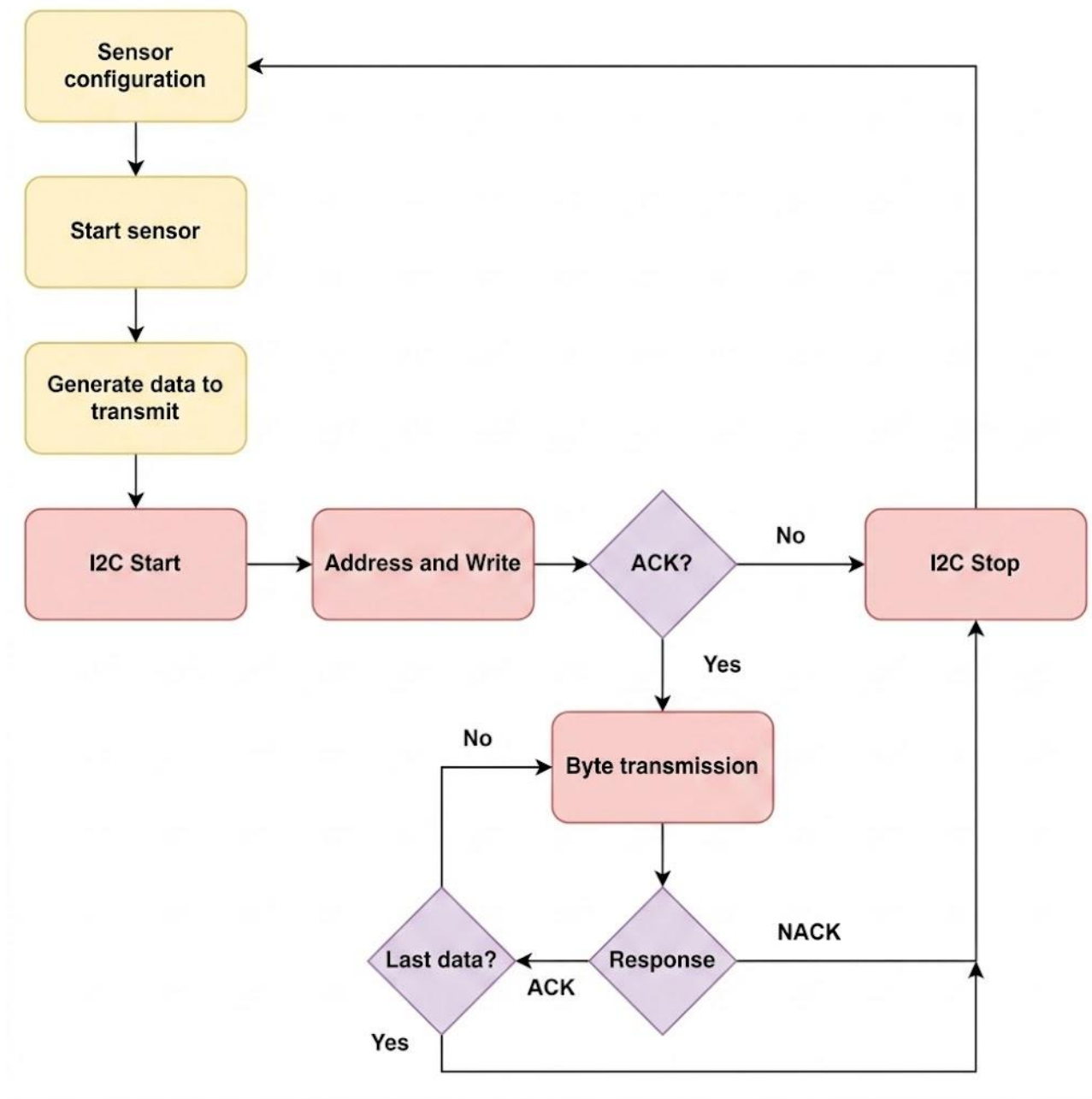


Figure 2. DUT functionality

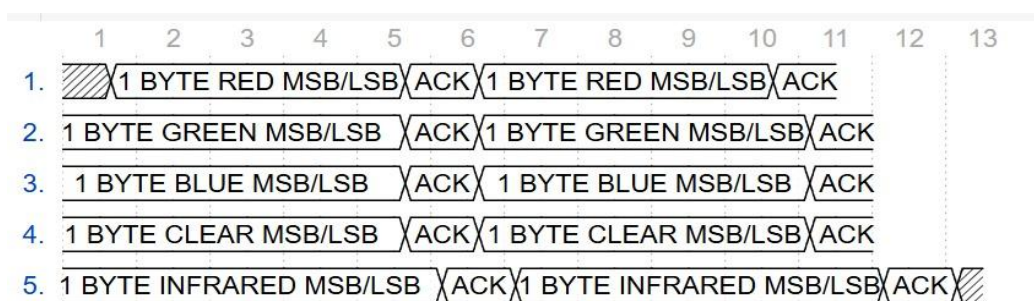


Figure 3. The order of data transmission on I2C

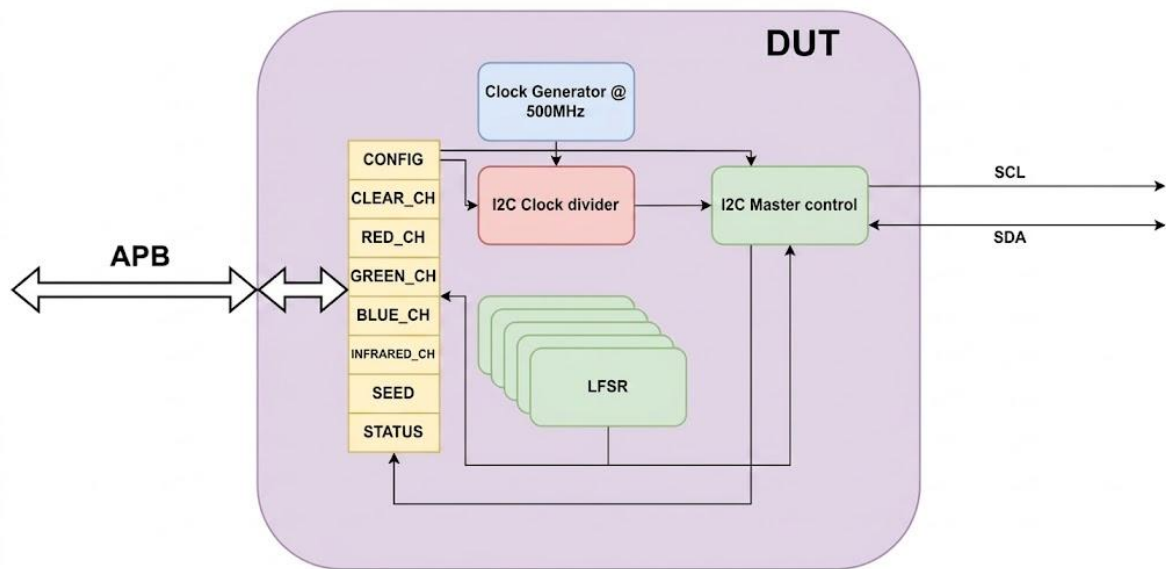
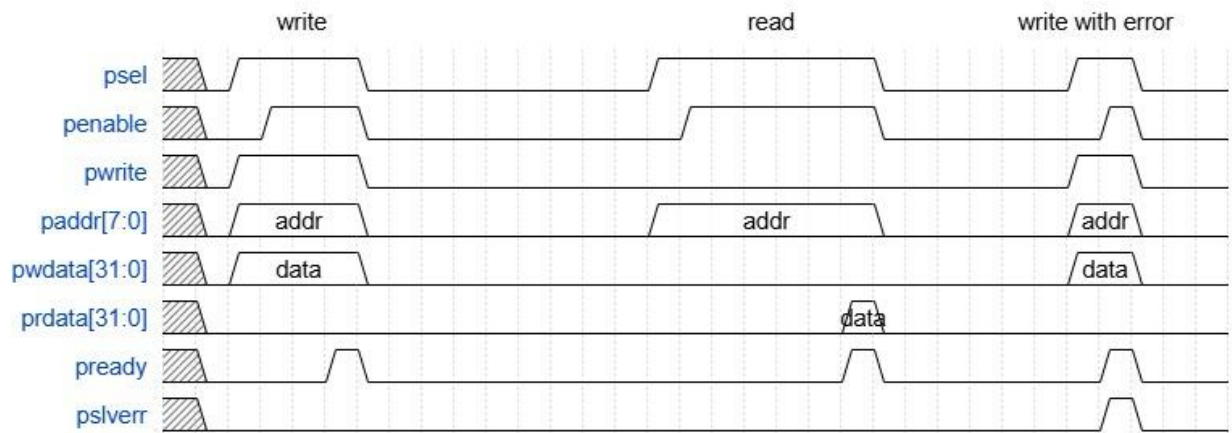


Figure 4. The schematic diagram of the DUT

1.5 SCHEMATIC DIAGRAM

Parameters:

- ADDR_WIDTH: The size of the APB address bus
- DATA_WIDTH: The size of the APB data bus



1.6 WAVEFORMS

Figure 5. Waveforms for the APB protocol

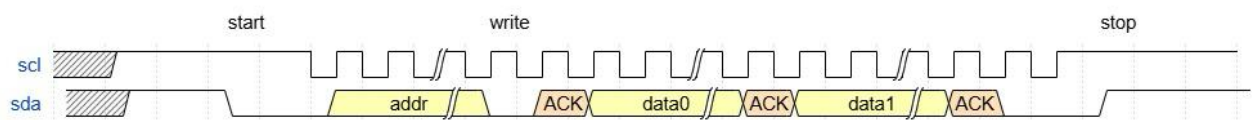


Figure 6. I2C waveforms for the I2C protocol (write)

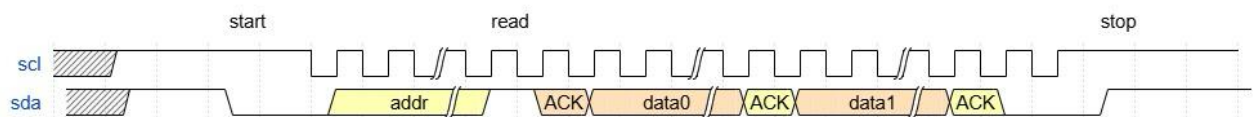


Figure 7. I2C waveforms for the I2C protocol (read)

1.7 REGISTER TABLE

Table 4. Table of the registers that can be accessed by the DUT

Register name	Access address	Register structure		Reset value	Description of the register fields
CONFIG	0x00			0x01	R/W configuration register
		Field name	Bit		
		FREQ	15		
			14		
		ADDR	13		
			12		
			11		
			10		
			9		
			8		
			7		
		ENDIAN	6		
		INFRARED_ON	5		
		BLUE_ON	4		
		GREEN_ON	3		
		RED_ON	2		

		<table><tr><td>CLEAR_ON</td><td>1</td></tr><tr><td>SD</td><td>0</td></tr></table>	CLEAR_ON	1	SD	0								
CLEAR_ON	1													
SD	0													
CLEAR_CH	0x02	Little Endian	0x00	Clear (transparent)										
RED_CH	0x04		<table><tr><td>15-8</td><td>7-0</td></tr></table>	15-8	7-0	0x00	Red color							
15-8	7-0													
GREEN_CH	0x06	<table><tr><td>LSB</td><td>MSB</td></tr></table>	LSB	MSB	0x00	Green color								
LSB	MSB													
BLUE_CH	0x08	Big Endian	0x00	Blue color										
INFRARED_CH	0x0C		<table><tr><td>15-8</td><td>7-0</td></tr></table>	15-8	7-0	0x00	Infrared light							
			15-8	7-0										
<table><tr><td>MSB</td><td>LSB</td></tr></table>	MSB	LSB												
MSB	LSB													
SEED	0x10	<table><tr><td>15-8</td><td>7-0</td></tr><tr><td>MSB</td><td>LSB</td></tr></table>	15-8	7-0	MSB	LSB	0xACE1	LFSR seed						
15-8	7-0													
MSB	LSB													
STATUS	0x12	<table><tr><td>Field name</td><td>Bit</td></tr><tr><td>RESERVED</td><td>15-3</td></tr><tr><td>NACK</td><td>2</td></tr><tr><td>BSY</td><td>1</td></tr><tr><td>STATUS</td><td>0</td></tr></table>	Field name	Bit	RESERVED	15-3	NACK	2	BSY	1	STATUS	0	0x00	Sensor status
Field name	Bit													
RESERVED	15-3													
NACK	2													
BSY	1													
STATUS	0													

• Configuration register

Field name	Bit	Description
FREQ	15	00 = 100 kHz; 01 = 400 kHz ; 10 = 1 MHz ; 11 = 3.4 MHz
	14	

ADDR_I2C	13	The address that must be accessed on I2C
	12	
	11	
	10	
	9	
	8	
	7	
ENDIAN	6	0 = LITTLE ENDIAN; 1 = BIG ENDIAN
INFRARED_ON	5	Turn on Infrared channel; 1 = power on; 0 = shutdown (default)
BLUE_ON	4	Turn on Blue channel; 1 = power on; 0 = shutdown (default)
GREEN_ON	3	Turn on Green channel; 1 = power on; 0 = shutdown (default)
RED_ON	2	Turn on Red channel; 1 = power on; 0 = shutdown (default)
CLEAR_ON	1	Turn on Clear channel; 1 = power on; 0 = shutdown (default)
SD	0	Shutdown setting; 0 = power on; 1 = shutdown

• Status register

Field name	Bit	Description
Reserved	15-3	-
NACK	2	0 = ACK; 1 = NACK (from I2C)
BSY	1	0 = Free; 1 = Busy
STATUS	0	0 = Off; 1 = On

Verification environment architecture
List of elements to be verified
Functional coverage elements
Verification scenarios and tests
Results obtained

2.1 VERIFICATION ENVIRONMENT ARCHITECTURE

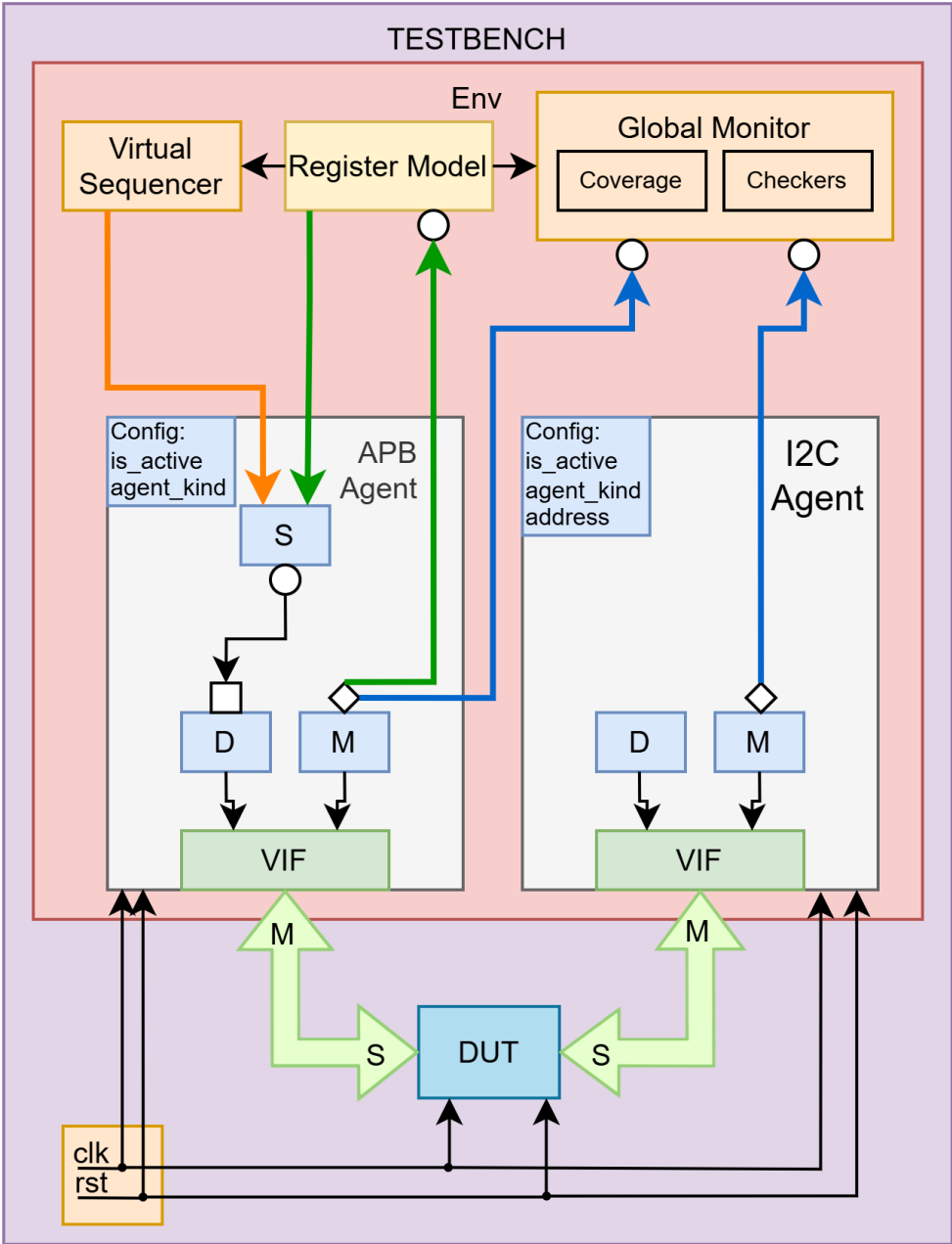


Figure 8. Example overview of the verification environment

2.1.1 Driver for the APB interface

The `apb_driver` class is an extension of the `uvm_driver` class, parameterized with the address and data widths (`APB_AW`, `APB_DW`). This class models a master-type driver for the APB interface.

In `build_phase` the virtual interface is initialized through `uvm_config_db`.

In `run_phase`, after waiting for the initial reset to pass, two processes are started that run in parallel inside a `join_any` fork.

One process calls the `wait_reset()` task, which waits to detect the assertion of the reset and initializes the signals for the APB interface.

The other process takes sequence items from the APB sequencer, calls the `drive_trans()` task, and then signals to the sequencer that the current transaction has finished and that the driver is ready to receive another.

The `drive_trans()` task, after waiting the number of clock cycles corresponding to the value of the delay field in the transaction, sets the address, asserts the `psel` signal and, depending on the operation type (write or read), drives the `pwrite` signal to 1 or 0. If it is a write operation, the data is also placed on `pwdata`. The task then waits one clock cycle and asserts the `penable` signal. It then waits for the `pready` signal to be asserted by the DUT, after which the control signals are deasserted.

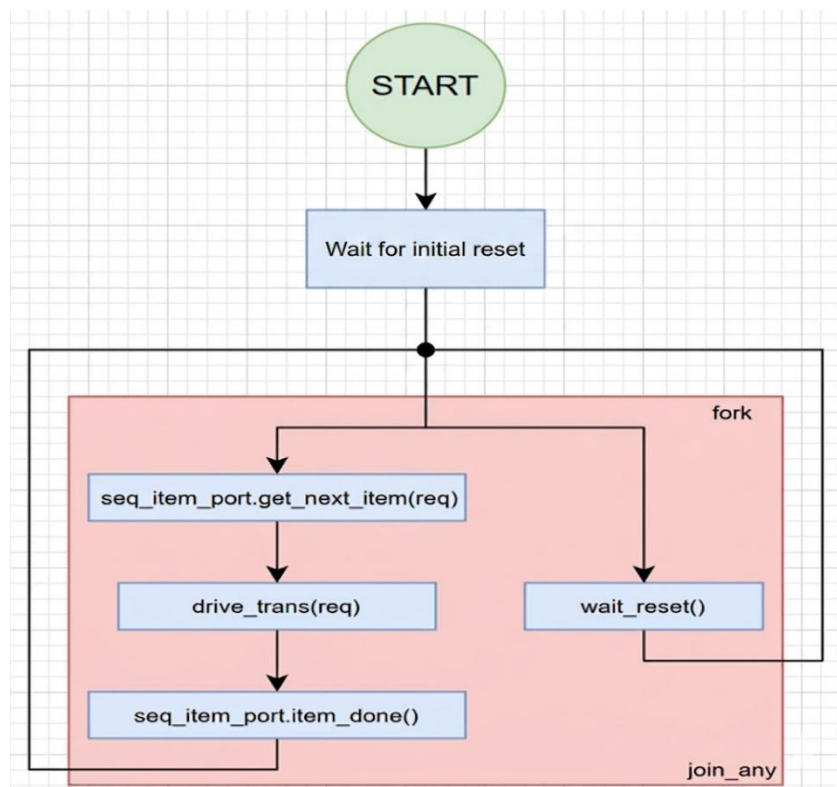


Figure 9. Logic flow diagram of the APB driver

2.1.2 Driver for the I2C interface

The I2C driver waits to detect the START condition, checks whether the received address equals the agent's address and whether the access is a write, and then sends the address response. If the address matches, after receiving each word it sends the data response until it detects the STOP condition.

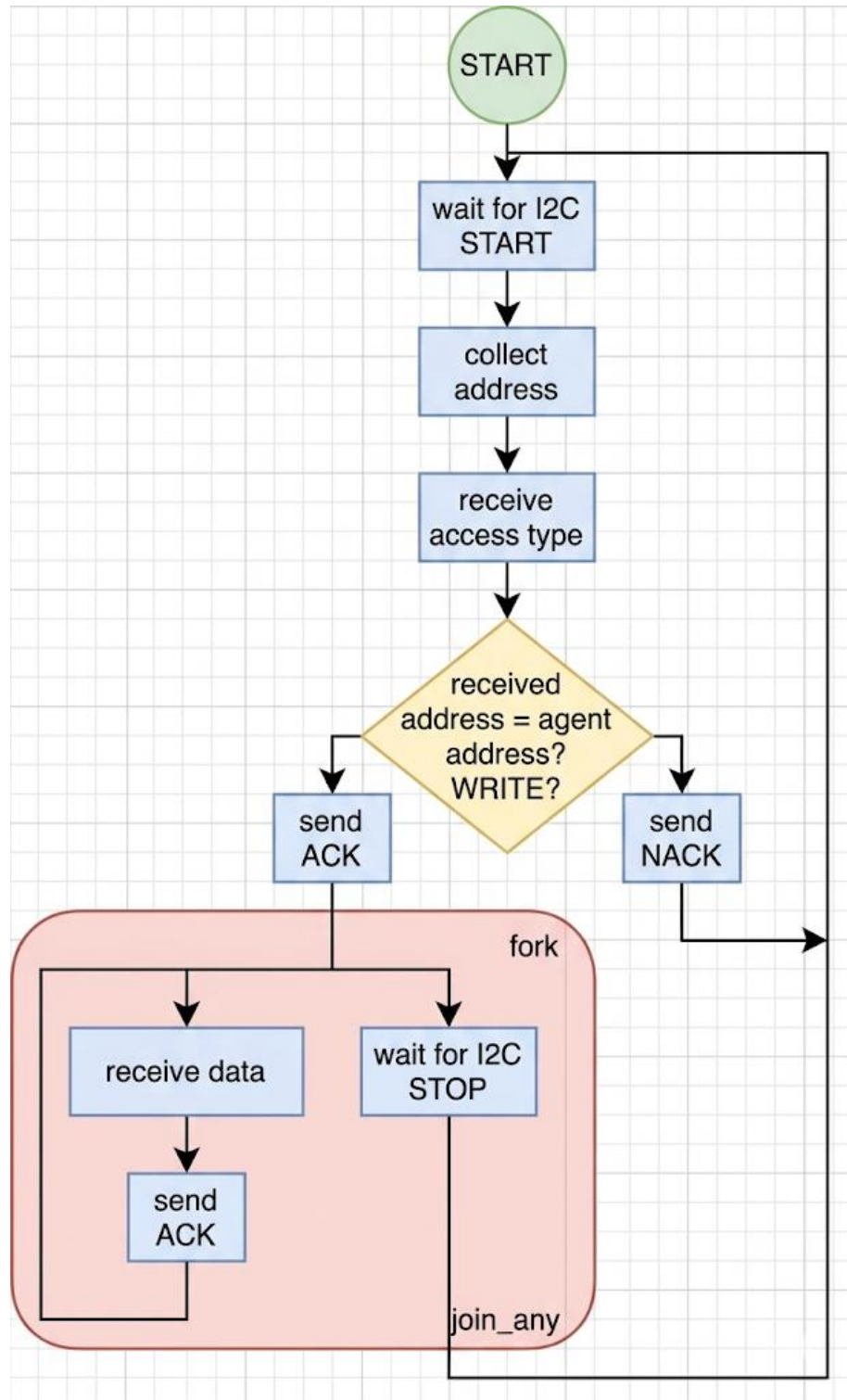


Figure 10. Logic flow diagram of the I2C driver

2.1.3 Scoreboard / reference model

The scoreboard handles the APB and I2C transactions, storing the data in the corresponding registers and verifying their integrity through assertions. It also collects coverage information to evaluate the degree of testing of the DUT registers. The scoreboard uses two functions, `write_apb` and `write_i2c`.

The `write_apb` function is used for processing APB transactions. APB transactions are used for configuring the registers and storing the read color data. Thus, this function uses assertions to verify whether the configuration data of the transactions matches the data in the registers.

The `write_i2c` function is used for processing I2C transactions. I2C transactions are used for transferring the data about the read colors. This function thus verifies whether the data transmitted over I2C is the same as the data in the respective color registers.

2.1.4 Monitor for the APB interface

The `apb_monitor` class is an extension of the `uvm_monitor` class, parameterized with the address and data widths (`APB_AW`, `APB_DW`). This class models a passive monitor for the APB bus, responsible for observing and collecting the transactions that take place on the APB interface, without influencing its behavior.

In `build_phase` the virtual interface is obtained from the UVM database, and if the interface is not set, execution is stopped with a fatal error.

In `run_phase` the monitor waits for the reset to be deasserted, then starts the parallel execution of three tasks using `fork join_any`.

The `collect_transactions()` task waits for the `psel`, `penable` and `pready` signals to be active simultaneously, the moment at which a transaction is considered valid. The address (`paddr`), the operation type (read/write depending on `pwrite`), the data (`pwdata` or `prdata`) and the error bit (`pslverr`) are extracted. The transaction is then forwarded through `apb_port`.

The `delay_counter()` task computes the delay between transactions. It monitors the `psel` signal and counts the clock cycles in which it is not active, in order to determine the delay between transactions. The delay value is then associated with the current transaction.

Another task started in parallel detects the assertion of the reset signal, and when it is active it stops the other parallel tasks.

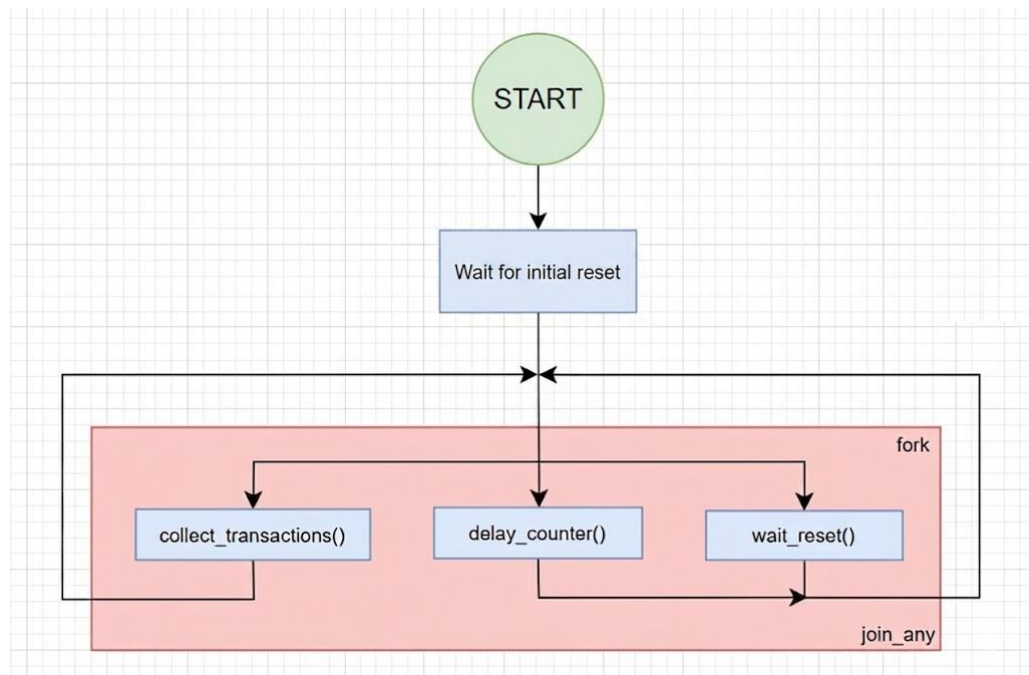


Figure 11. Logic flow diagram of the APB monitor

2.1.5 Monitor for the I2C interface

The I2C monitor watches the I2C interface and waits to detect the START condition. It collects the transaction until it detects the STOP condition and sends it through a port to the Scoreboard and the I2C coverage collector.

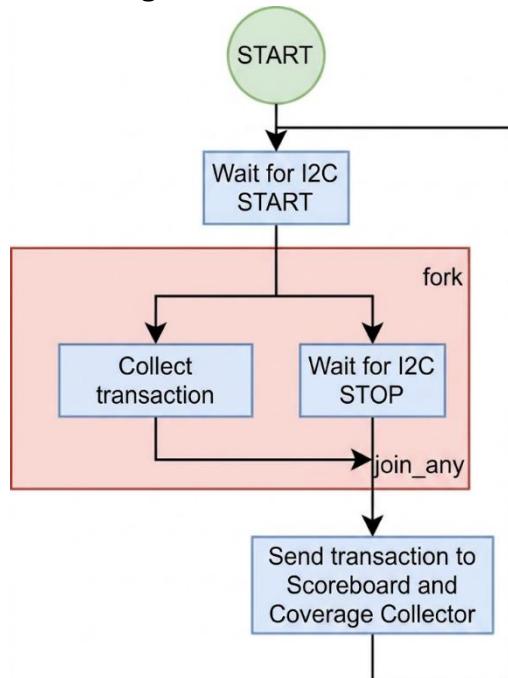


Figure 12. Logic flow diagram of the I2C monitor

2.1.6 Transaction for the APB interface

Table 5. The fields of the APB transaction

Field name	Data type	Size	Values	Description
addr	bit	`APB_AW-1	[0:1F]	APB address
data	bit	`APB_DW-1		Transmitted data
access	access_kind	1	{READ, WRITE}	Access type
trans_delay	int	32	[1:5]	Delay between transactions
slv_error	bit	1	[0:1]	Error signal from the DUT

`APB\_AW and `APB_DW are two defines declared in the file apb_defines.sv and serve to increase the configurability of the APB UVC. If the user wishes to change the size of the data bus or the address bus, this can be done very easily by simply changing the value of those two defines, without needing to replace the value in all the UVC files.

The default value of the address bus is 5 bits, and that of the data bus is 32 bits.

2.1.7 Transaction for the I2C interface

Table 6. The fields of the I2C transaction

Field name	Data type	Size	Values	Description
addr	bit	7	[0:7F]	I2C address
access	access_kind	1	{READ, WRITE}	Access type
nr_words	int	32	10	Number of words
data	bit	10x8b		Data vector
addr_resp	resp_kind	1	{ACK, NACK}	Address response
data_resp	resp_kind	1	{ACK, NACK}	Response of the last data word

2.1.8 Agent for the APB interface

The APB agent is an active agent, the APB having the role of Master in this project. It contains the driver, the monitor and the sequencer for APB.

2.1.9 Agent for the I2C interface

The I2C agent has the role of Slave, being reactive. It contains the I2C Driver, the I2C Monitor and the I2C coverage collector. It receives the I2C address from the verification environment and passes it to the Driver.

2.1.10 Coverage collector for the I2C interface

Coverpoint	Values	Description
addr_cov	[0:127]	Transactions are performed with all addresses
addr_resp_cov	ACK, NACK	Both types of address response are received
data_resp_cov	ACK, NACK	Both types of data response are received
data_cov	[0:0xFF]	All data bits toggle

2.1.11 Coverage collector for the scoreboard / reference model

Coverpoint	Values	Description
reg_config[15:14]	00, 01, 10, 11	Checks the operating-frequency values of the DUT
reg_config[13:7]	[0:127]	Checks the addresses used for performing I2C transfers
reg_config[6]	0,1	Checks the Little Endian / Big Endian setting
reg_config[5]	0,1	Checks the infrared channel state (on/off)
reg_config[4]	0,1	Checks the blue channel state (on/off)
reg_config[3]	0,1	Checks the green channel state (on/off)
reg_config[2]	0,1	Checks the red channel state (on/off)
reg_config[1]	0,1	Checks the clear channel state (on/off)
reg_config[0]	0,1	Checks the DUT state (on/off)

clear_ch	[0:127]	Checks the values received on the Clear channel
red_ch	[0:127]	Checks the values received on the Red channel
green_ch	[0:127]	Checks the values received on the Green channel
blue_ch	[0:127]	Checks the values received on the Blue channel
infrared_ch	[0:127]	Checks the values received on the Infrared channel
status[2]	ACK, NACK	Checks the reception of ACK/NACK
status[1]	0, 1	Checks the Available/Busy state
status[0]	0,1	Checks the register on/off state

The scoreboard coverage collector monitors the input combinations sent to the DUT in order to build a coverage report. Several coverage points (coverpoints) were defined:

- A coverage point for the configuration register, with all its fields
- A coverage point for each color channel, including clear and infrared
- A coverage point for the status register, with all its fields
- A coverage point for the seed register

2.2 LIST OF ELEMENTS TO BE VERIFIED

Table 5. The assertions on the APB interface

Checker	Description
x_z_state_check	The signals will never be x or z
stable_signal_check	The signals do not change before pready
penable_check	penable is not asserted without psel

Table 6. The assertions on the I2C interface

Checker	Description
x_z_state_check	The signals will never be x or z
sda_stable_check	SDA does not change while SCL is asserted
scl_full_cycle_check	SCL always completes a full cycle
scl_stable_check	SCL does not change outside of a transfer

Table 7. The checkers in the scoreboard

Checker	Description
write_apb	Checks the DUT configuration register through an APB transfer
write_i2c	Checks the correct execution of the data transfer over I2C

2.3 FUNCTIONAL COVERAGE ELEMENTS

Table 6. The functional coverage elements for the data on the output interface

Event: cover_trans_done_ev - emitted at the end of a burst transaction	
Cover Point	Description
reg_config[15:14]	Checks the operating-frequency values of the DUT
reg_config[13:7]	Checks the addresses used for performing I2C transfers
reg_config[6]	Checks the Little Endian / Big Endian setting
reg_config[5]	Checks the infrared channel state (on/off)
reg_config[4]	Checks the blue channel state (on/off)
reg_config[3]	Checks the green channel state (on/off)
reg_config[2]	Checks the red channel state (on/off)
reg_config[1]	Checks the clear channel state (on/off)
reg_config[0]	Checks the DUT state (on/off)
clear_ch	Checks the values received on the Clear channel

red_ch	Checks the values received on the Red channel
green_ch	Checks the values received on the Green channel
blue_ch	Checks the values received on the Blue channel
infrared_ch	Checks the values received on the Infrared channel
status[2]	Checks the reception of ACK/NACK
status[1]	Checks the Available/Busy state
status[0]	Checks the register on/off state

2.4 VERIFICATION SCENARIOS AND TESTS

Table 7. The list of scenarios designed for verifying the DUT

Scenario title	Description of the proposed scenario
register reset	The register values will be read immediately after the reset signal was asserted, in order to verify their value
register access	All registers are written with random values, and then read back
LFSR	Writing the seed register with different values
transmission order	Changing the data transmission order (Little Endian, Big Endian)
clock frequency	Testing the correct data transmission with different frequency values
ON/ OFF	Disabling the color channels, verification on the I2C interface

Table 8. The list of tests implemented for verifying the DUT

Name of the file containing the test	Description of the test performed
test_registers.sv	In this test we verify the Read/Write functionalities of the registers.
test_functional.sv	In this test we verify whether the I2C protocol works correctly.

2.5 RESULTS OBTAINED

The three tests were run in the Xilinx Vivado simulator (XSim, UVM 1.2). All passed with 0 errors and 0 warnings.

Running test_base.sv (5 random APB transactions):

- APB transactions captured: 5 / 5
- APB functional coverage: 41.67% | I2C functional coverage: 0.00%

Running test_registers.sv (read defaults → random write → read-back):

- APB transactions captured: 24 / 24
- APB functional coverage: 72.22% | I2C functional coverage: 0.00%

Running test_functional.sv (configuration + continuous I2C transmission):

- APB transactions captured: 12 / 12
- I2C transactions verified: 1 / 1 (address 0x29, WRITE, 10 data bytes, ACK + valid STOP)
- APB functional coverage: 20.83% | I2C functional coverage: 46.88%

All protocol assertions on the APB and I2C interfaces (3 APB + 4 I2C checkers) passed without failures throughout every simulation. I2C coverage is 0% in test_base and test_registers because these tests do not activate the sensor; the I2C data path is exercised by test_functional.